

AI Employee Knowledge Assistant

Architecture Design, Application Flows & 9-Day Hackathon Task Breakdown

Sprint: 9 working days | Tasks: 27 | Effort: 58 hours | Task limit: each task under 4 hours

1. Project Overview

The AI Employee Knowledge Assistant is a role-aware Retrieval-Augmented Generation (RAG) chatbot. Employees ask questions about internal company documents and receive answers grounded in documents they are authorized to access, with supporting source references.

The sprint delivers a working chatbot, not an enterprise knowledge-management platform. Mandatory capabilities are employee authentication, PDF/DOCX ingestion, role-filtered retrieval, conversational question answering, source references, fallback responses, and simple document replacement.

ID	Core requirement	Scope
FR01	Employee authentication	Employee login and backend-derived role
FR02	Document ingestion	Administrator upload of PDF and DOCX files
FR03	Role-based access	Filter retrieval by authenticated user permissions
FR04	RAG question answering	Use authorized retrieved document context
FR05	Conversation history	Recent messages for follow-up questions
FR06	Source references	Show supporting document and section
FR07	Fallback behavior	Avoid answers when authorized evidence is insufficient
FR08	Document updates	Replace an outdated document and its indexed chunks

2. Technology Stack

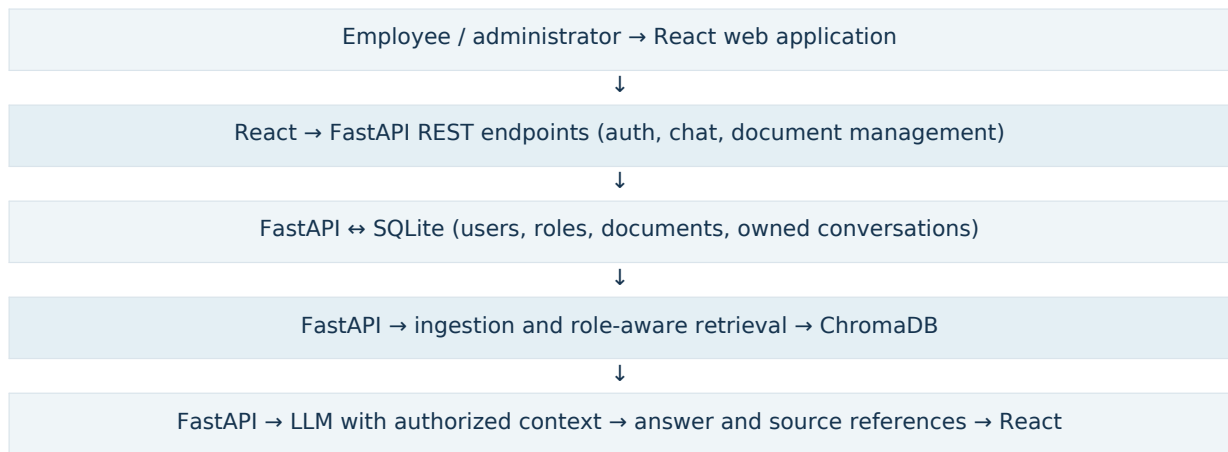
Component	Technology	Purpose
Frontend	React.js	Login, chat, source display, admin upload
Backend	Python FastAPI	Auth, RBAC, APIs, document processing and RAG orchestration
Application data	SQLite	Users, roles, document records and conversation history
Vector database	ChromaDB	Chunk storage, embeddings, role-filtered semantic search
LLM	Ollama local model or external API	Generate grounded answers from authorized context
Embedding model	Local model or embedding API	Create vectors for documents and questions
Document parsing	Python PDF/DOCX libraries	Extract and chunk document text

Scope note: No Ruby on Rails, enterprise SSO, agents, advanced version lifecycle, or separate workflow engine.

3. System Architecture Design

3.1 High-level components

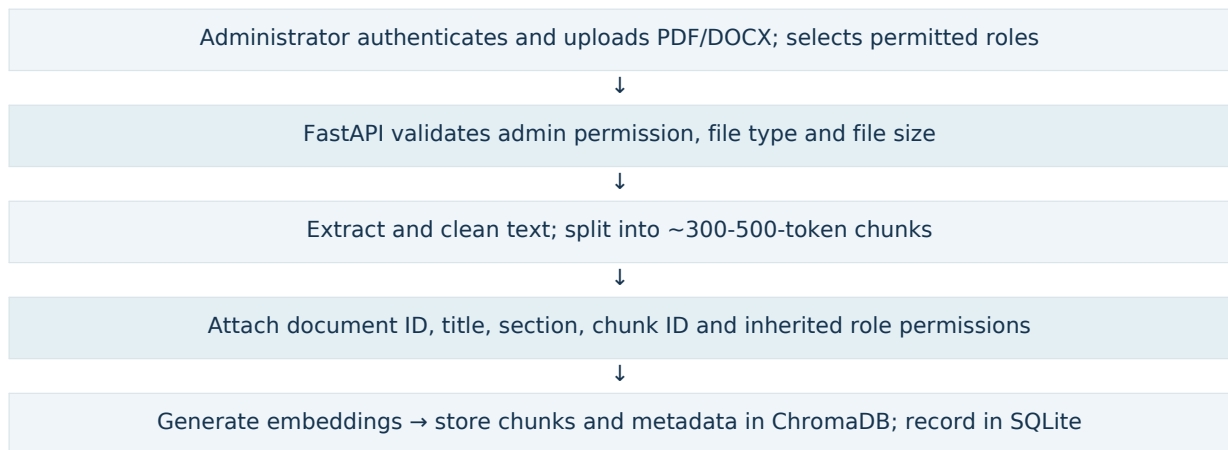
Component interaction



Frontend: Presents login, chat, citations, and a minimal administrator upload screen. FastAPI: Validates sessions, derives roles, enforces document permissions, manages uploads, runs retrieval and calls the LLM. SQLite: Stores application records. ChromaDB: Stores searchable chunks plus permission metadata. LLM: Generates answers; it does not decide document access.

3.2 Document ingestion architecture

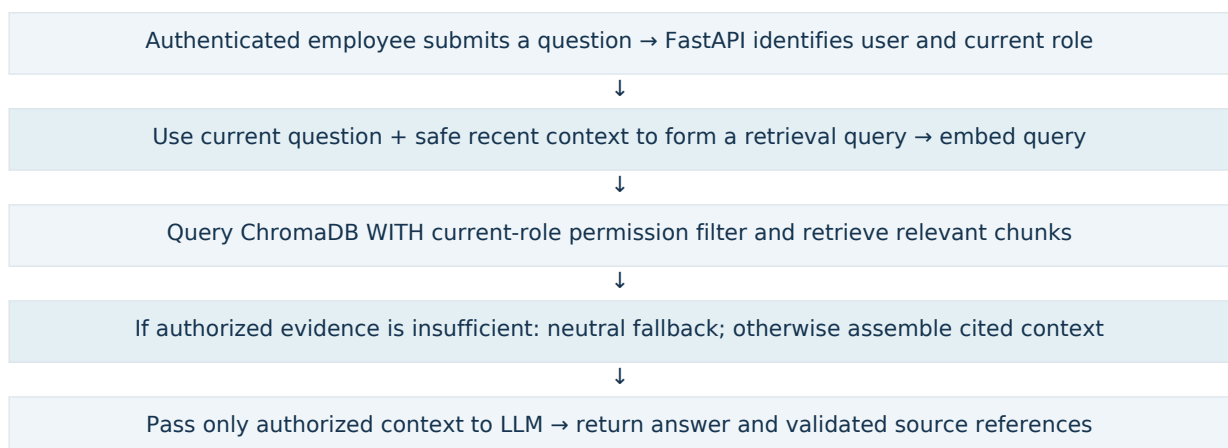
Administrator document indexing



ChromaDB metadata example: document_id=DOC_001; document_name=HR Policy; section=Employee Benefits; chunk_id=CHUNK_001; allow_employee=false; allow_hr=true; allow_manager=true. The fields are scalar values compatible with metadata filtering.

3.3 Role-aware RAG architecture

Question-answering pipeline



Security invariant: The permission filter is part of the ChromaDB retrieval query; restricted chunks must never enter prompt construction. Do not trust a role supplied in the frontend request. Only return references that correspond to authorized retrieved chunks.

3.4 Minimal RBAC model

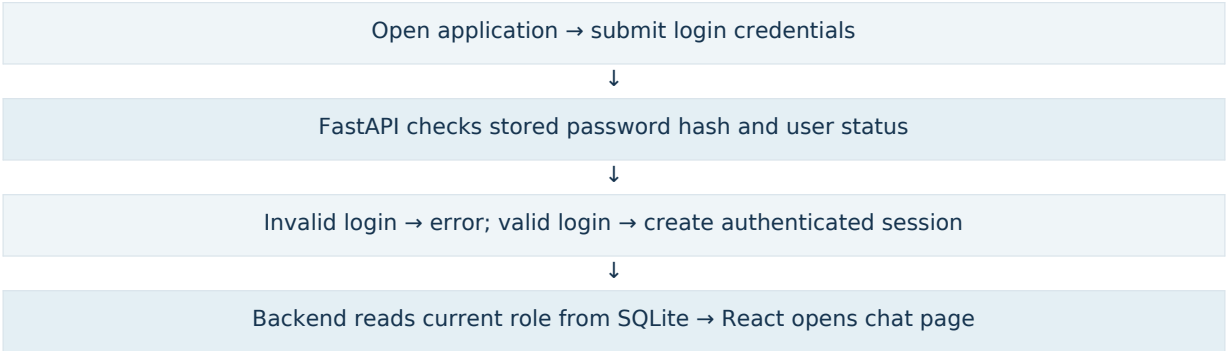
Role	Document access
Employee	General employee documents
HR	General documents and HR-authorized documents
Manager	General documents and manager-authorized documents
Administrator permission	Upload/replace documents and assign role permissions; separate from employee role

A document may be accessible to multiple roles. Every chunk inherits its parent document permissions. SQLite holds the document permission record; the same permissions are copied to ChromaDB metadata for filtered retrieval.

4. Application Flow Design

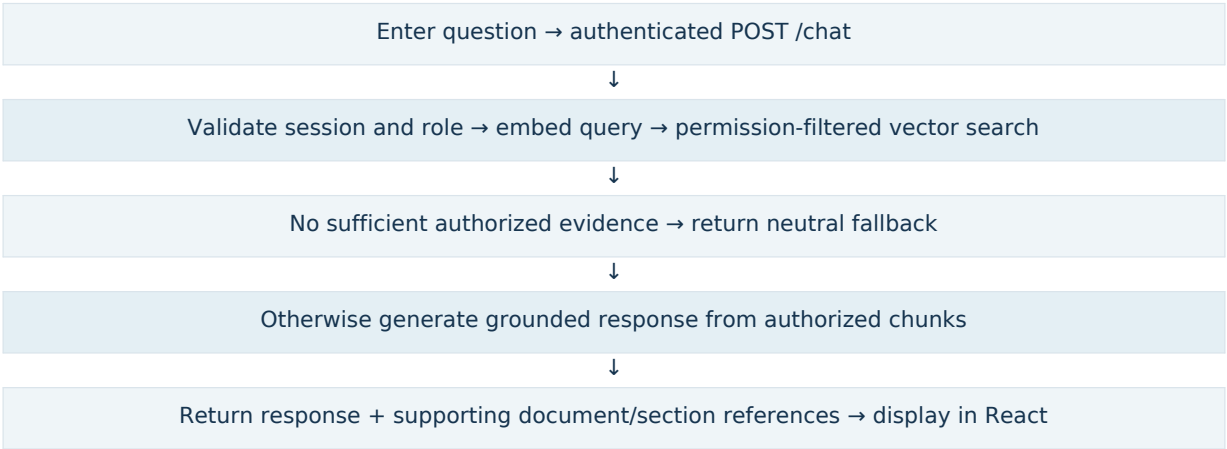
4.1 Employee login

Login sequence



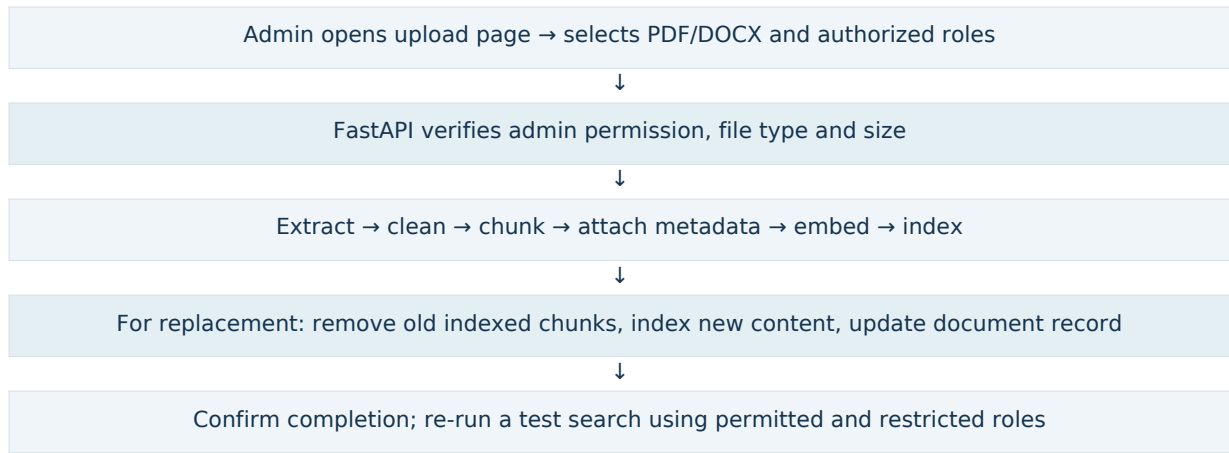
4.2 Employee chat

Chat sequence



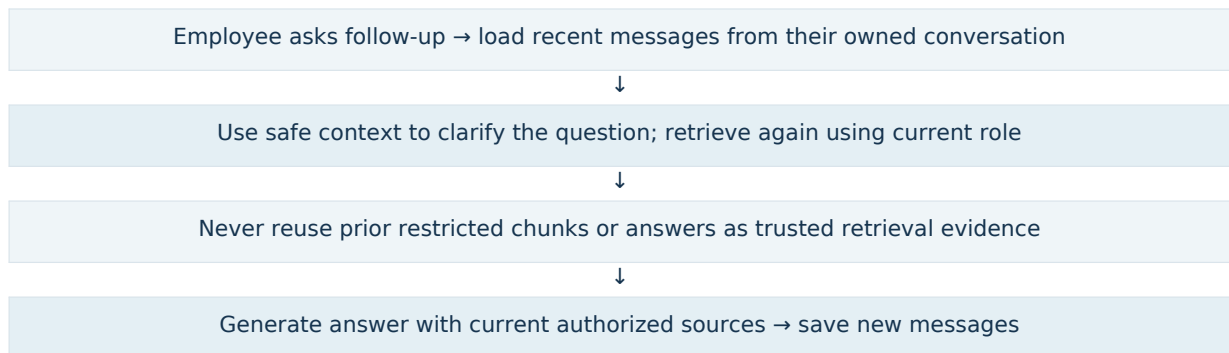
4.3 Administrator document upload / update

Document management sequence



4.4 Follow-up conversation

Conversation sequence



5. Sprint Plan at a Glance

Phase	Focus	Days	Effort
1	Project setup and RBAC foundation	1	6.5 h
2	Document processing and role-aware RAG	2-4	19.5 h
3	Conversational AI and source references	5-7	19.5 h
4	Security testing and finalization	8-9	12.5 h
TOTAL	27 tasks; each < 4 hours	9	58 h

6. Detailed Task Breakdown

Each task is a discrete piece of work with a tangible completion check. Time estimates are planning assumptions for a team familiar with the chosen stack.

Phase 1 - Project Setup & RBAC Foundation

Day 1 - Application initialization | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T01	Initialize project and FastAPI backend	Create repo, FastAPI app, environment configuration and /health endpoint.	2 h	API starts; /health returns success.
T02	Set up React frontend	Initialize React; add a basic chat layout; connect to a test API endpoint.	2 h	React can send a request and display the response.
T03	Implement authentication and roles	Create SQLite users with hashed passwords; add login, sessions and Employee/HR/Manager demo roles.	2.5 h	Login works; backend identifies the authenticated role.

Phase 2 - Document Processing & Role-Aware RAG

Day 2 - Document ingestion | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T04	Implement administrator document upload	Build admin-protected PDF/DOCX upload endpoint with type and size validation.	2 h	Admin can upload; non-admin/invalid files are rejected.
T05	Implement text extraction	Extract readable text from representative PDF/DOCX files with Python libraries.	2.5 h	Test PDFs and DOCX files produce usable text.
T06	Implement document chunking	Clean content; split into ~300-500-token chunks and attach document, section and chunk IDs.	2 h	Every chunk includes usable source metadata.

Day 3 - Embeddings and permissions | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T07	Configure embedding model	Set up local embeddings or API; generate vectors for documents and questions.	2 h	Document and query vectors use the same model.
T08	Configure ChromaDB	Store chunks, vectors, IDs and source metadata; test similarity search.	2 h	Indexed chunks are returned for a sample query.
T09	Implement permission metadata	Define per-document allowed roles; copy scalar access metadata to every indexed chunk.	2.5 h	Every chunk has the expected role flags.

Day 4 - Role-filtered retrieval | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T10	Implement role-filtered semantic search	Read role from authenticated session; include matching permission filter in ChromaDB query.	2.5 h	Only authorized chunks are returned by retrieval.
T11	Implement document replacement	Remove old chunk IDs, index replacement content and update its document record.	2 h	Old chunks no longer appear after update.
T12	Test retrieval permissions	Exercise Employee, HR and Manager queries against general and restricted documents.	2 h	Test users see only the chunks assigned to their roles.

Phase 3 - Conversational AI & Source References

Day 5 - RAG answer generation | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T13	Configure LLM integration	Connect FastAPI to Ollama or the chosen API and test a completion.	2 h	Backend successfully receives an LLM response.
T14	Implement RAG question answering	Retrieve authorized chunks; build a grounding prompt; return an answer to the question.	2.5 h	Answers use authorized retrieved context only.
T15	Implement source references	Return document/section references derived from retrieved chunk metadata.	2 h	References correspond to real authorized chunks.

Day 6 - Conversation and fallback | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T16	Implement conversation history	Persist recent messages for the owning user; use relevant history for follow-ups.	2.5 h	A follow-up question is interpreted in context.
T17	Implement fallback responses	Handle no/weak authorized matches and context that cannot support an answer.	2 h	Chatbot acknowledges unavailable information.
T18	Secure conversation context	Restrict history by owner; recheck current role and avoid replaying unauthorized source content.	2 h	Other users and changed-role users cannot bypass access.

Day 7 - Frontend integration | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T19	Integrate frontend login	Wire login and authenticated chat requests to FastAPI; handle expiry.	2 h	A logged-in user can chat under their own role.
T20	Improve chat interface	Add history display, loading states, errors and readable source references.	2.5 h	Employee can read answers and supporting sources.
T21	End-to-end functionality testing	Walk through login, retrieval, answer, follow-up and source display.	2 h	Full employee flow works with demo documents.

Phase 4 - Security Testing & Finalization

Day 8 - RBAC and RAG validation | 6.5 h

ID	Task	Implementation details / deliverable	Time	Done when...
T22	Test RBAC and bypass attempts	Check role isolation, forged role input and that prohibited chunks never reach LLM prompts.	2.5 h	No unauthorized source content is sent to the LLM.
T23	Test answer quality and citations	Check ~10-15 representative questions against documents and cited sections.	2 h	Record and correct unsupported answers/references.
T24	Test fallback and edge cases	Check absent content, restricted questions, expired session and replaced documents.	2 h	Safe fallback and update behavior work as intended.

Day 9 - Final demo | 6 h

ID	Task	Implementation details / deliverable	Time	Done when...
T25	Fix critical issues and tune performance	Resolve prioritized bugs; tune retrieval size/context and response path.	2 h	Critical chat, citation and access defects addressed.
T26	Prepare demonstration data	Upload general/restricted docs and configure Employee, HR and Manager accounts.	2 h	Demo corpus and role-specific accounts work.
T27	Final verification and presentation	Run full app and rehearse allowed/denied retrieval, follow-ups, citations and fallback.	2 h	Stable end-to-end hackathon demonstration.

7. Suggested API Design

Method	Endpoint	Purpose / access
POST	/auth/login	Authenticate employee; issue session
GET	/auth/me	Return authenticated user and current role
POST	/chat	Role-aware RAG answer; authenticated user
GET	/conversations	List only the current user's conversations
GET	/conversations/{id}	Read conversation only when owned by requester
POST	/documents/upload	Upload/index PDF/DOCX; administrator only
GET	/documents	List only documents accessible to current user
PUT	/documents/{id}	Replace document and indexed content; administrator only

8. Basic Database Design

Table	Fields / purpose
users	user_id, username, password_hash, role, is_admin
documents	document_id, document_name, file_path, allowed_roles, uploaded_at
conversations	conversation_id, user_id, created_at
messages	message_id, conversation_id, role, content, created_at

ChromaDB stores each chunk's document ID, title, section, chunk ID and Boolean role permission flags alongside text and embeddings. Authorization is evaluated against the current authenticated user when each retrieval request is made.

9. Risk Control and Mitigation

Risk	Impact	Mitigation
Unauthorized document access	Restricted content reaches a user	Use role filters within vector retrieval before building the LLM prompt.
Unsupported answers	Incorrect company guidance	Ground answers in authorized context; attach verified sources; fall back when unsupported.
Outdated content	Old document details surface	Remove the replaced document's earlier chunks and reindex the new content.
Irrelevant retrieval	Answers use unrelated sections	Tune chunking, embedding choices and top-k based on sample questions.
Missing/inaccessible answer	No authorized evidence is available	Return neutral fallback without disclosing restricted documents.
Conversation permission bypass	Prior messages leak newly restricted context	Check ownership and current permissions; do not reuse unverified history as evidence.
Slow responses	Poor chat usability	Limit retrieved context and choose a model suited to available hardware.

10. Sprint Progress Tracking

Day	Milestone	Completion check
1	Application foundation	Frontend/backend connected; authenticated role available.
2	Document processing	PDF/DOCX extracted and chunked.
3	Vector index	Embeddings and permission metadata stored.
4	Role-aware retrieval	Only permitted chunks returned by filtered search.
5	Core RAG chatbot	Grounded answers with real source references.
6	Conversation support	Follow-ups, fallback and safe history.
7	Frontend integration	Complete login-to-answer UI flow.
8	Validation	RBAC, answer support and edge cases tested.
9	Final demonstration	Critical defects fixed; demo scenarios verified.

11. Final Deliverables and Success Criteria

The Day 9 deliverable is a running React + FastAPI employee chatbot with SQLite authentication, administrator PDF/DOCX ingestion, ChromaDB role-filtered retrieval, an LLM-backed answer flow, short conversation history, source references, fallback responses and simple document replacement.

Successful demo: An Employee, HR user and Manager ask the same representative questions; each receives only information authorized for their role. A follow-up question uses conversation context without bypassing permissions. A missing-information question produces a fallback, and every source reference identifies an actual permitted document section.

Outside this sprint: Ruby on Rails, enterprise SSO, complex RBAC management UI, advanced policy lifecycle/versioning, automated HR workflows, dedicated guidance engines and formal enterprise benchmarking.